The trace is a loop, and boxes slide on it

**One Mathlib diff, one string diagram, one Lean proof —
and where all of this sits in category theory and in Szabó's linear algebra**

Abstract

This note is about a single theorem of Mathlib, `Matrix.trace_units_conj'`, and the one-line diff that gave it a docstring. The claim it makes — $\mathrm{tr}(M^{-1}NM) = \mathrm{tr}\,N$ — is the exact algebraic content of one diagrammatic move: *the trace is a closed wire, and a box may be slid around a closed wire.* We draw the move, we run it forwards to obtain the theorem, we put the Lean proof next to the picture step by step, we identify the move in category theory (dinaturality of the trace in a compact closed / traced monoidal category), and we locate the corresponding statements in L. Szabó, *Bevezetés a lineáris algebrába.* Everything asserted here about Lean is backed by the machine-checked file `RequestProject/TraceLoop.lean` of the accompanying development, which builds with no `sorry` and no added axioms.

Contents

1 The question, and the short answer

The diff under discussion is

```
-set_option linter.docPrime false in
+/-- The trace is invariant under conjugation by a unit, variant of
+Matrix.trace_units_conj with the inverse on the left.  -/
 -- TODO(.../issues/6607):  fix elaboration so that the ascription isn't needed
 theorem trace_units_conj' (M : (Matrix m m R)ˣ) (N : Matrix m m R) :
   trace ((↑M⁻¹ :  Matrix _ _ _) * N * (↑M : Matrix _ _ _)) = trace N :=
```

So the diff itself is documentation only: the primed name used to be excused from the `docPrime` linter by a `set_option`, and is now excused because it has a real docstring. *No mathematics changed.* What the new docstring names, though, is precisely the interesting thing: the primed variant is the *other side of the loop*.

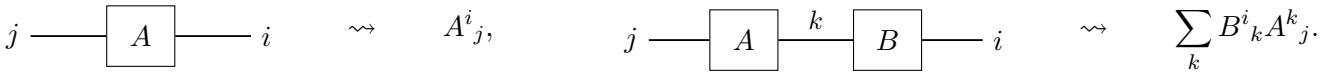
Is this theorem related to exactly that move, proof and picture? Yes, and rather tightly:

- The theorem $\operatorname{tr}(M^{-1}NM) = \operatorname{tr} N$ is the diagram equation obtained by sliding the box M once around the closed wire until it meets M^{-1} (Section 4).
- The unprimed sibling $\operatorname{tr}(MNM^{-1}) = \operatorname{tr} N$ is the *same picture*; the two Lean statements differ only in where you cut the circle open to write it as a linear string of symbols. That is exactly why the primed lemma's proof term is one application of the unprimed one.
- The Mathlib proof of the unprimed lemma is `rw [trace_mul_cycle, Units.inv_mul, one_mul]`: one slide, one cancellation, one unit law — the three moves of the picture, in order (Section 5).

2 The trace is a loop

2.1 Boxes and wires

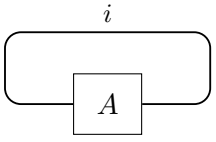
In string-diagram (Penrose) notation a matrix $A \in \mathbb{K}^{m\times n}$ is a box with one input leg and one output leg; an index sits on each leg. Joining two legs means summing over the shared index — that, and nothing else, is contraction:



In Lean this reading of the product is literally definitional: `Diagram.penrose_contraction` says `(A * B) i j = ∑ k, A i k * B k j` and its proof is `rfl`.

2.2 Closing the wire

Now take one box and join its *output* leg back to its *input* leg. There is no free index left; the diagram is a closed loop with a box on it, and the join sums over the one remaining index:



$$= \qquad \sum_i A^i{}_i \;=\; \operatorname{tr} A.$$

That is the whole of the slogan "tr is a loop". The trace is not an extra piece of notation bolted onto matrices: it is what the language produces when you close the only wire there is. In Lean:

```
theorem trace_is_a_loop (A : Matrix n n R) : trace A = ∑ i, A i i := rfl
```

2.3 A bare loop counts dimensions

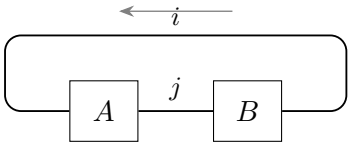
Put no box at all on the closed wire. The empty box is the identity matrix, i.e. the Kronecker delta $\delta^i{}_j$, and closing it gives $\sum_i \delta^i{}_i$:

$$= \quad \sum_i \delta^i{}_i \;=\; n.$$

`TraceLoop.loop_eq_dim`: `trace (1 : Matrix n n R) = Fintype.card n`. A closed wire is therefore not "nothing": it is the dimension of the space the wire carries. (This is the source of the factors of n that appear from nowhere in tensor computations, and of dim appearing in the Euler characteristic in the categorical version, Section 7.)

3 Sliding boxes on the loop

Put two boxes on one closed wire, $A \in \mathbb{K}^{m\times n}$ and $B \in \mathbb{K}^{n\times m}$:



$$= \quad \sum_i \sum_j A^i{}_j B^j{}_i.$$

Two indices are contracted now: the wire *between* the boxes (j), and the wire that *closes* the loop (i). Nothing in the picture says which of A, B comes first — a circle has no beginning. Cutting the circle open just to the left of A reads the picture as $\mathrm{tr}(AB)$; cutting it just to the left of B reads the same picture as $\mathrm{tr}(BA)$. Hence

Theorem 3.1 (Sliding, `TraceLoop.slide_box`)**.** $\mathrm{tr}(AB) = \mathrm{tr}(BA)$ *for all* $A \in \mathbb{K}^{m\times n}$, $B \in \mathbb{K}^{n\times m}$.

What "sliding" means at the level of indices. The picture hides exactly two bookkeeping steps, and it is worth seeing them, because the Lean proof in `TraceLoop.lean` is written to expose them:

$$\mathrm{tr}(AB) = \sum_i \sum_j A^i{}_j B^j{}_i \overset{(1)}{=} \sum_j \sum_i A^i{}_j B^j{}_i \overset{(2)}{=} \sum_j \sum_i B^j{}_i A^i{}_j \;=\; \mathrm{tr}(BA).$$

Step (1) is `Finset.sum_comm`: the picture does not say in which order to contract the two closed indices. Step (2) is `mul_comm` in the scalar ring: the two boxes sit at unrelated places on the wire, so their entries are just numbers being multiplied. *Commutativity of the scalars is where the move is paid for* — see Fact 3.4.

Corollary 3.2 (Full cyclic invariance)**.** *For square boxes* $A_1, \ldots, A_k$ *on one loop,* $\mathrm{tr}(A_1 A_2 \cdots A_k) = \mathrm{tr}(A_2 \cdots A_k A_1)$*, and hence the trace is invariant under every cyclic rotation.*

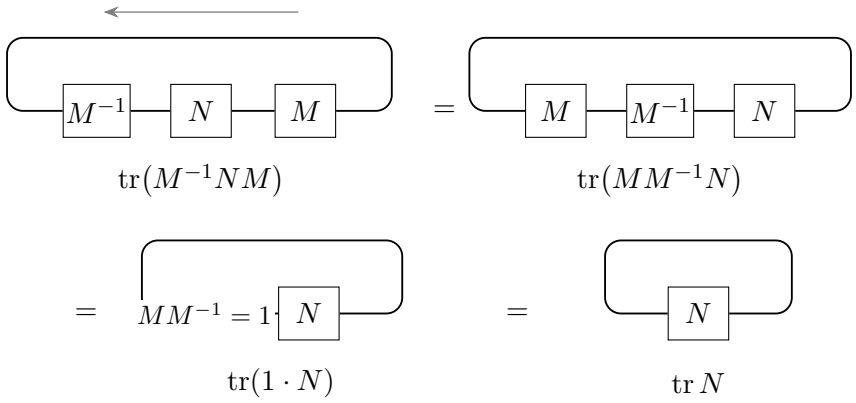
In Lean this is stated for a *list* of boxes, which is the honest formalisation of "boxes threaded onto a loop": `TraceLoop.trace_rotate` (`trace l.prod = trace (l.rotate 1).prod`) and `TraceLoop.trace_rotate_iterate` for k clicks.

Cool fact 3.3 (Cyclic, not symmetric). You may rotate the boxes; you may *not* permute them. On a circle, turning ABC into BAC would require lifting B over A, i.e. making the two wires cross. And indeed $\operatorname{tr}(ABC) \neq \operatorname{tr}(BAC)$ in general: with $A = \left(\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right)$, $B = \left(\begin{smallmatrix} 0 & 0 \\ 1 & 0 \end{smallmatrix}\right)$, $C = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right)$ one gets $\operatorname{tr}(ABC) = 1$ but $\operatorname{tr}(BAC) = 0$. The picture predicts the failure before the computation does: planar isotopy of the disc allows rotation of the circle, not transposition of points on it. (Machine-checked as `TraceLoop.trace_cyclic_not_symmetric`.)

Cool fact 3.4 (Where commutativity hides). Over a non-commutative ring R the move is false: for $A = (a)$, $B = (b)$ in $R^{1\times 1}$, $\operatorname{tr}(AB) = ab$ and $\operatorname{tr}(BA) = ba$. Diagrammatically, a non-commutative ring of scalars means the "empty" region of the plane is no longer commutative, and boxes can no longer be moved past each other freely. This is why the Mathlib statements live under `[NonUnitalCommSemiring R]` / `[CommSemiring R]`, not under `[Semiring R]`.

4 The theorem in the diff, as a picture

Now put three boxes on the loop: M^{-1}, N, M. This is the left-hand side of `trace_units_conj'`.



Three moves, in order:

1. **Slide.** M travels once round the closed wire and lands in front of M^{-1}. Nothing was created or destroyed; we merely re-cut the circle.
2. **Annihilate.** Adjacent inverse boxes fuse: $MM^{-1} = 1$. This is the only step that uses invertibility, and it is a purely local, "two boxes in series" rewrite.
3. **Absorb.** An identity box is a bare piece of wire and is erased.

Remark 4.1 (The primed and unprimed statements are one picture). The unprimed `trace_units_conj` is $\operatorname{tr}(MNM^{-1}) = \operatorname{tr} N$. Cut the very same three-box circle open one box further along and you read $\operatorname{tr}(M^{-1}NM)$ instead. So the two Mathlib lemmas are two *linearisations of one diagram*, and Mathlib's proof of the primed one reflects this exactly: it is not a new argument but the unprimed lemma applied to M^{-1}. What the new docstring in the diff adds is precisely the sentence a reader of the picture would want: "variant … with the inverse on the left".

Remark 4.2 (Only one-sided invertibility is needed). The annihilation step only ever uses $MM^{-1} = 1$ on *one* side. Hence the sharper statement `TraceLoop.trace_conj_of_mul_eq_one`: if $PQ = 1$ then $\operatorname{tr}(QAP) = \operatorname{tr} A$, with no assumption that Q is a two-sided inverse and no `Units` bundling. Diagrammatically obvious; a small strengthening of the library statement.

5 How the Lean proof works, line by line

5.1 Mathlib's proof

```
theorem trace_mul_cycle (A : Matrix m n R) (B : Matrix n p R) (C : Matrix p m
R) :
trace (A * B * C) = trace (C * A * B) := by
rw [trace_mul_comm, Matrix.mul_assoc]

theorem trace_units_conj (M : (Matrix m m R)ˣ) (N : Matrix m m R) :
trace (↑M * N * ↑M⁻¹) = trace N := by
rw [trace_mul_cycle, Units.inv_mul, one_mul]

theorem trace_units_conj' (M : (Matrix m m R)ˣ) (N : Matrix m m R) :
trace (↑M⁻¹ * N * ↑M) = trace N :=
trace_units_conj M⁻¹ N
```

Read the three rewrites of the middle proof against the three moves of Section 4:

picture	Lean rewrite	what it does
slide a box round the loop	`trace_mul_cycle`	$\mathrm{tr}(MN \cdot M^{-1}) = \mathrm{tr}(M^{-1}MN)$
adjacent inverses fuse	`Units.inv_mul`	$M^{-1}M \rightsquigarrow 1$
an identity box is bare wire	`one_mul`	$1 \cdot N \rightsquigarrow N$
re-cut the circle elsewhere	`trace_units_conj M⁻¹ N`	gives the primed variant

The correspondence is exact, one tactic per move, in the same order. That is the sense in which the wire diagram is not an illustration of the proof but *is* the proof, written in a two-dimensional syntax.

5.2 The companion development

`RequestProject/TraceLoop.lean` re-derives all of this from the loop picture rather than from the library, so that each diagrammatic notion has a name. The slide, in particular, is proved from the double sum, so the two halves of the argument in Section 3 are visible:

```
theorem two_boxes_on_a_loop (A : Matrix m n R) (B : Matrix n m R) :
trace (A * B) = ∑ i, ∑ j, A i j * B j i := by
simp [Matrix.trace, Matrix.diag, Matrix.mul_apply]

theorem slide_box (A : Matrix m n R) (B : Matrix n m R) :
trace (A * B) = trace (B * A) := by
rw [two_boxes_on_a_loop, two_boxes_on_a_loop, Finset.sum_comm]
exact Finset.sum_congr rfl fun j _ => Finset.sum_congr rfl fun i _ => mul_comm
- -
```

Contents of the file:

Lean name (namespace `TraceLoop`)	statement
`trace_is_a_loop`	$\operatorname{tr} A = \sum_i A^i{}_i$ (closing the wire)
`loop_eq_dim`	bare loop = dim
`two_boxes_on_a_loop`	$\operatorname{tr}(AB) = \sum_i \sum_j A^i{}_j B^j{}_i$
`slide_box`	$\operatorname{tr}(AB) = \operatorname{tr}(BA)$
`slide_three`	$\operatorname{tr}(ABC) = \operatorname{tr}(C(AB))$
`trace_cyclic_not_symmetric`	rotation yes, permutation no (counterexample)
`trace_rotate`, `trace_rotate_iterate`	rotation of a list of boxes on the loop
`trace_units_conj_left`	$\operatorname{tr}(MNM^{-1}) = \operatorname{tr} N$
`trace_units_conj_right`	$\operatorname{tr}(M^{-1}NM) = \operatorname{tr} N$ (*the diff's theorem*)
`trace_conj_of_mul_eq_one`	$PQ = 1 \Rightarrow \operatorname{tr}(QAP) = \operatorname{tr} A$
`Similar`, `Similar.refl/symm/trans`	Szabó Def. 4.4 and Thm. 4.5
`trace_similar`, `det_similar`, `charpoly_similar`	similarity invariants
`trace_basis_independent`	$\operatorname{Tr}_N(e\, f\, e^{-1}) = \operatorname{Tr}_M(f)$ for $e : M \simeq N$
`trace_eq_matrix_trace_of_basis`	endomorphism trace = matrix trace
`trace_comp_slide`	$\operatorname{Tr}(g \circ f) = \operatorname{Tr}(f \circ g)$, no indices
`trace_isProj_eq_finrank`	a loop through a projection is its rank

Earlier files of the same development already contain the two diagram lemmas quoted in the discussion, `Diagram.loop_eq_dim` and `Diagram.trace_cyclic` in `DiagramDictionary.lean`, and the exam move `Pattern.trace_conj` in `ExamPatterns.lean`.

6 Big ideas

6.1 Why the trace has a picture at all, and the determinant does not (quite)

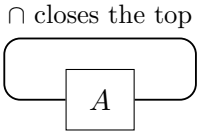
The determinant is invariant under conjugation too, and Szabó proves it in one line by multiplicativity: $|X^{-1}AX| = |X^{-1}|\,|A|\,|X| = |A|\,(|X^{-1}||X|) = |A|$. That proof is *scalar juggling*: it moves numbers past each other. For the trace no such route exists, because tr is emphatically not multiplicative ($\operatorname{tr}(AB) \neq \operatorname{tr} A \operatorname{tr} B$ in general). The trace instead has the property that no other linear functional has:

Cool fact 6.1 (Uniqueness). Up to a scalar, tr is the *only* linear functional $f : \mathbb{K}^{n\times n} \to \mathbb{K}$ with $f(AB) = f(BA)$ for all A, B. Equivalently, the space of such functionals is one-dimensional, the quotient of $\mathbb{K}^{n\times n}$ by the span of all commutators $AB - BA$ being one-dimensional.

So cyclicity is not one property of the trace among many — it *is* the trace. And cyclicity is exactly what a closed wire expresses. The picture therefore carries the full defining content of the invariant, which is why conjugation invariance falls out of it with no computation at all.

6.2 The loop as a pairing: bending wires

Where does the closing wire come from? From the canonical pairing. Write $\cup$ (*cup*, coevaluation) for the state $\mathbb{K} \to V \otimes V^*$, $1 \mapsto \sum_i e_i \otimes e^i$, and $\cap$ (*cap*, evaluation) for $V^* \otimes V \to \mathbb{K}$, $\varphi \otimes v \mapsto \varphi(v)$. Then

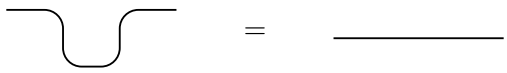
$\operatorname{tr} A \;=$ 

$= \cap \circ (A \otimes \mathrm{id}_{V^*}) \circ \cup.$

The two facts that make wires bendable are the *snake* (zig-zag) equations

$$(\cap \otimes \mathrm{id}) \circ (\mathrm{id} \otimes \cup) = \mathrm{id}, \qquad (\mathrm{id} \otimes \cap) \circ (\cup \otimes \mathrm{id}) = \mathrm{id},$$

drawn as pulling a bent wire straight:



In matrix terms, bending a leg transposes the box; that is `Diagram.transpose_is_adjoint` in the development, $\langle Ax, y\rangle = \langle x, A^\top y\rangle$.

6.3 One theorem, three invariants

Sliding proves more than the trace statement. The same three-move argument, with "slide" replaced by the appropriate cyclic property, gives the standard family of similarity invariants:

$$\operatorname{tr}(X^{-1}AX) = \operatorname{tr} A, \qquad |X^{-1}AX| = |A|, \qquad r(X^{-1}AX) = r(A), \qquad f_{X^{-1}AX} = f_A.$$

The last is the strongest: the characteristic polynomial contains both the trace (coefficient of x^{n-1}, up to sign) and the determinant (constant term). In Lean, `TraceLoop.charpoly_similar`; in the diagram language, $\operatorname{tr}(A^k)$ for all k is a family of loop diagrams (a loop with k copies of A on it), and by Newton's identities these determine the characteristic polynomial in characteristic 0.

Cool fact 6.2 (Loops count). $\operatorname{tr}(A^k)$ counts closed walks of length k in the graph whose adjacency matrix is A — literally the number of ways of threading a closed wire through k boxes. Combinatorics reads the same picture as tensor calculus.

Cool fact 6.3 (Rank of a projection). If $P^2 = P$ then $\operatorname{tr} P$ is the rank of P, read as an element of $\mathbb{K}$: a loop through an idempotent box counts the dimension it projects onto — consistent with the bare loop counting the whole dimension, which is the case $P = 1$. Lean: `TraceLoop.trace_isProj_eq_finrank`, from Mathlib's `LinearMap.IsProj.trace`.

7 The category-theoretic way to think about it

7.1 String diagrams are the syntax of monoidal categories

A string diagram is a morphism in a monoidal category: side-by-side juxtaposition is $\otimes$, wiring in series is $\circ$, a bare wire is id, and the coherence theorem for monoidal categories (Mac Lane) is what licenses us to draw pictures at all rather than carry associators around. For $\mathbb{K}$-linear algebra the relevant category is $\mathbf{FdVect}_{\mathbb{K}}$: objects are finite-dimensional spaces, morphisms are linear maps, $\otimes$ is the tensor product.

7.2 Compact closedness: why closing a wire is legal

$\mathbf{FdVect}_{\mathbb{K}}$ is *compact closed*: every object V has a dual V^* with unit $\cup : \mathbf{1} \to V \otimes V^*$ and counit $\cap : V^* \otimes V \to \mathbf{1}$ satisfying the snake equations. Finite dimensionality is exactly what is needed for $\cup$ to exist. In such a category every endomorphism $f : V \to V$ has a scalar

$$\operatorname{Tr}(f) \;=\; (\mathbf{1} \xrightarrow{\cup} V \otimes V^* \xrightarrow{f\otimes \mathrm{id}} V \otimes V^* \xrightarrow{\sigma} V^* \otimes V \xrightarrow{\cap} \mathbf{1}) \;\in\; \operatorname{End}(\mathbf{1}) = \mathbb{K},$$

where σ is the symmetry that swaps the two factors; this is the categorical trace. Then:

- **Sliding = dinaturality.** For $f : V \to W$ and $g : W \to V$, $\operatorname{Tr}(g \circ f) = \operatorname{Tr}(f \circ g)$. This is one of the axioms of a *traced monoidal category* (Joyal–Street–Verity), where it is called *dinaturality* or *sliding*; in a compact closed category it is a theorem rather than an axiom. Lean: `TraceLoop.trace_comp_slide`, i.e. Mathlib's `LinearMap.trace_comp_comm'`.

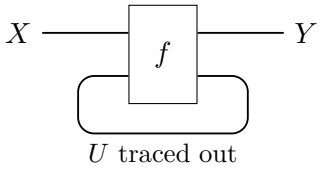
- **The diff's theorem = dinaturality once.** Take $f = M^{-1}N$ and $g = M$; then $\mathrm{Tr}(M^{-1}NM) = \mathrm{Tr}(g \circ f) = \mathrm{Tr}(f \circ g) = \mathrm{Tr}(M^{-1}MN) = \mathrm{Tr}(N)$. So `trace_units_conj'` is literally one instance of the sliding axiom followed by the fact that M is invertible.

- **Vanishing, superposing, yanking.** The other traced-category axioms are the remaining obvious moves: tracing out two wires one at a time equals tracing out their tensor product (vanishing); a trace next to an untouched diagram is a trace (superposing); and yanking a bent wire straight is the snake equation. $\mathrm{Tr}(\mathrm{id}_V) = \dim V$ is the bare loop.

- **Basis independence.** $\mathrm{End}(V) \cong V \otimes V^*$ and Tr is the evaluation map; no basis occurs anywhere. Conjugation invariance of the matrix trace is then a corollary: matrices of one endomorphism in two bases are related by conjugation, and the endomorphism trace does not know about the choice. Lean: `TraceLoop.trace_basis_independent` ($\mathrm{Tr}_N(e\,f\,e^{-1}) = \mathrm{Tr}_M(f)$ for an isomorphism e) together with `trace_eq_matrix_trace_of_basis`.

7.3 Partial trace: the same loop, on one of two wires

Categorically the trace is really *partial*: given $f : X \otimes U \to Y \otimes U$, close only the U wire.



$\rightsquigarrow \qquad \mathrm{Tr}_U(f) : X \to Y.$

This is the feedback loop of automata and of Bainbridge's flow networks, the partial trace of quantum information, and the reason traced monoidal categories were introduced in the first place. "Sliding a box on the loop" is then the statement that feedback does not care where along the feedback wire you insert a transformation, provided you insert its inverse elsewhere on the same wire.

7.4 Where graphical linear algebra proper differs

Graphical linear algebra (Paweł Sobociński's *graphicallinearalgebra.net*) works not in **FdVect** but in the prop of *linear relations*: a diagram $m \to n$ denotes a subspace of $\mathbb{K}^m \times \mathbb{K}^n$, not a map. The accompanying Lean development takes this as its official semantics (`GraphicalLinearAlgebra.lean`: `LinearRel R M N = Submodule R (M × N)`). Two consequences for the present story:

- Relations compose in both directions, so every diagram can be reflected; cups and caps are then *definable* from the copy/add generators rather than postulated, and the compact closed structure is free. Bending a wire is the transpose (`Diagram.transpose_is_adjoint`); mirroring the copy junction gives the add junction (a theorem in `DiagramDictionary.lean`).

- Closing a loop on a general relation is *not* a scalar — it is a subspace of $\mathbb{K}^0 \times \mathbb{K}^0$, so "tr" is only a number on the sub-prop of graphs of maps. The trace story therefore lives in the map fragment, which is exactly the fragment that matrices span.

8 Where to find this in Szabó László's book

Reference: L. Szabó, *Bevezetés a lineáris algebrába*, Polygon jegyzettár, Bolyai Intézet, Szeged, 2nd ed. 2006.

An honest caveat. The trace (*nyom*) does not appear in this booklet at all; the word does not occur in the text. What *does* appear, and in the exact shape of the theorem in the diff, is the conjugation-invariance pattern, applied to the other invariants. The relevant places are:

place	Hungarian	content
§2, Def. 2.4, Thm. 2.5	Mátrixok	the matrix product $\sum_k a_{ik}b_{kj}$ and its associativity — the contraction rule of Section 2
§4, Def. 4.3	elfajuló	singular / non-singular ($\lvert A\rvert = 0$ or not)
§4, Def. 4.4	hasonló	$A \approx B$ iff $B = X^{-1}AX$ for non-singular X — *the exact left-hand side of* `trace_units_conj'`
§4, Thm. 4.5	—	$\approx$ is reflexive, symmetric, transitive, and *similar matrices have equal determinant*; the proof is $\lvert X^{-1}AX\rvert = \lvert X^{-1}\rvert\lvert A\rvert\lvert X\rvert = \lvert A\rvert$
§9, Thm. 9.6	rang	$r(AB) \le r(A), r(B)$; multiplication by an invertible matrix preserves rank; *similar matrices have equal rank*
§13, Thm. 13.4	Áttérés új bázisra	$A_{\mathcal{E}',\mathcal{F}'} = P^{-1}A_{\mathcal{E},\mathcal{F}}S$: change of bases conjugates the matrix
§13, Cor. 13.5	—	for a linear *transformation*, $A_{\mathcal{E}'} = P^{-1}A_{\mathcal{E}}P$: *the matrices of one transformation in different bases are similar*
§14, Thm. 14.3	karakterisztikus polinom	similar matrices have the same characteristic polynomial

Two remarks worth making to a reader of the booklet.

- Corollary 13.5 is the *reason* the theorem in the diff matters: the expression $X^{-1}AX$ is never arbitrary, it is "the same map, seen in another basis". An invariant of similarity is a property of the map, not of the array of numbers. The categorical statement of Section 7 says the same thing without choosing bases at all.
- Theorem 4.5's determinant argument and the trace argument are genuinely different proofs of parallel statements: the determinant slides because it is multiplicative into a commutative ring; the trace slides because it is cyclic. Only the trace argument is the loop. If one wanted to add the trace to the booklet, the natural home is immediately after Theorem 4.5, as a second invariant, with the proof "$\operatorname{tr}(X^{-1}AX) = \operatorname{tr}((AX)X^{-1}) = \operatorname{tr} A$".

9 Summary

picture	algebra	Lean
close a wire	$\operatorname{tr} A = \sum_i A^i{}_i$	`TraceLoop.trace_is_a_loop`
bare loop	$\operatorname{tr} 1 = n$	`TraceLoop.loop_eq_dim`
slide a box round the loop	$\operatorname{tr}(AB) = \operatorname{tr}(BA)$	`TraceLoop.slide_box`
rotate k boxes	cyclic invariance	`TraceLoop.trace_rotate_iterate`
slide, annihilate, absorb	$\operatorname{tr}(M^{-1}NM) = \operatorname{tr} N$	`TraceLoop.trace_units_conj_right` `Matrix.trace_units_conj'`
same picture, cut elsewhere	$\operatorname{tr}(MNM^{-1}) = \operatorname{tr} N$	`Matrix.trace_units_conj`
cup and cap, snake equation	compact closed structure	`Diagram.transpose_is_adjoint`
loop without indices	$\operatorname{Tr}(gf) = \operatorname{Tr}(fg)$	`TraceLoop.trace_comp_slide`

Verdict on the opening question. The theorem in the diff is exactly the move "slide a box around the closed wire until it meets its inverse", its Mathlib proof is exactly that move followed by cancellation and unit absorption, and its primed/unprimed pair is exactly the choice of where to cut the circle open. The diff itself changed no mathematics — it replaced a linter suppression by the docstring that names which cut is which.